

DOCUMENTAÇÃO TÉCNICO-CIENTÍFICA DE EVIDÊNCIAS

Projeto Integrador Etapa 1

ClinSecure
Autenticação e Gestão de Credenciais

DOCUMENTAÇÃO TÉCNICO-CIENTÍFICA DE EVIDÊNCIAS

Projeto Integrador Etapa 1

Sumário

1. Identificação do Projeto
2. Escopo da Primeira Etapa
3. Arquitetura da Autenticação
4. Requisitos 1.1 a 1.12
5. Evidências Funcionais
6. Evidências do Banco de Dados
7. Evidências de Código
8. Auditoria e Logs
9. Sessões e Logout
10. Histórico de Desenvolvimento
11. Kanban
12. Repositório GitHub
13. Release
14. Checklist Final
15. Checklist da Entrega
16. Pendências Antes da Entrega

1. Identificação do Projeto

Nome do projeto: ClinSecure.

Objetivo: Plataforma web desenvolvida para o gerenciamento seguro de pesquisas clínicas e dos dados de seus participantes, garantindo que o acesso seja restrito a usuários autorizados e protegendo a identidade dos indivíduos.

Tecnologias utilizadas: Back-end utilizando Python 3.14.6 e Django 5.2.17; Front-end com HTML5, CSS3, Bootstrap 5 e JavaScript; Banco de dados MySQL 8.0.46; Versionamento via Git e GitHub.

Repositório GitHub: CauaPeresAdriani/Projeto-Integrador

Integrantes: Cauã Peres Adriani e Luis Poli.

2. Escopo da Primeira Etapa

Esta documentação corresponde à primeira etapa do Projeto Integrador. O escopo avaliado nesta fase é "Autenticação e Gestão de Credenciais". De acordo com as instruções oficiais (documento "ENTREGAS"), os membros devem evidenciar o funcionamento prático de todos os itens implementados por meio do Front-end da aplicação, justificar as lógicas na camada de segurança e garantir o armazenamento seguro de senhas.

3. Arquitetura da Autenticação

O ClinSecure implementa uma arquitetura de segurança rigorosa dividida nas seguintes etapas:

Usuário e Senha: O fluxo é iniciado com o usuário fornecendo as credenciais na tela de login.

Validação das credenciais e Armazenamento Temporário: O sistema busca o usuário no banco de dados e verifica bloqueios ativos. Se aprovado, a senha é validada através do algoritmo Argon2.

Verificação do TOTP: O usuário é levado à rota `/verificar_2fa/` onde fornece um token de 6 dígitos gerado em seu aplicativo autenticador.

Autenticação definitiva: Se o token for validado, a sessão final é estabelecida e o usuário é redirecionado à área protegida.

Proteção e Auditoria: Todas as ações, como validações bem-sucedidas de senhas, falhas de 2FA e bloqueios temporários por tentativas excedidas, são registradas no banco de dados para rastreabilidade, e bloqueios são aplicados em caso de repetição de falhas (Rate Limit).

4. Requisitos 1.1 a 1.12

1.1 Uso de hash criptográfico seguro para senhas (Argon2, bcrypt ou PBKDF2)

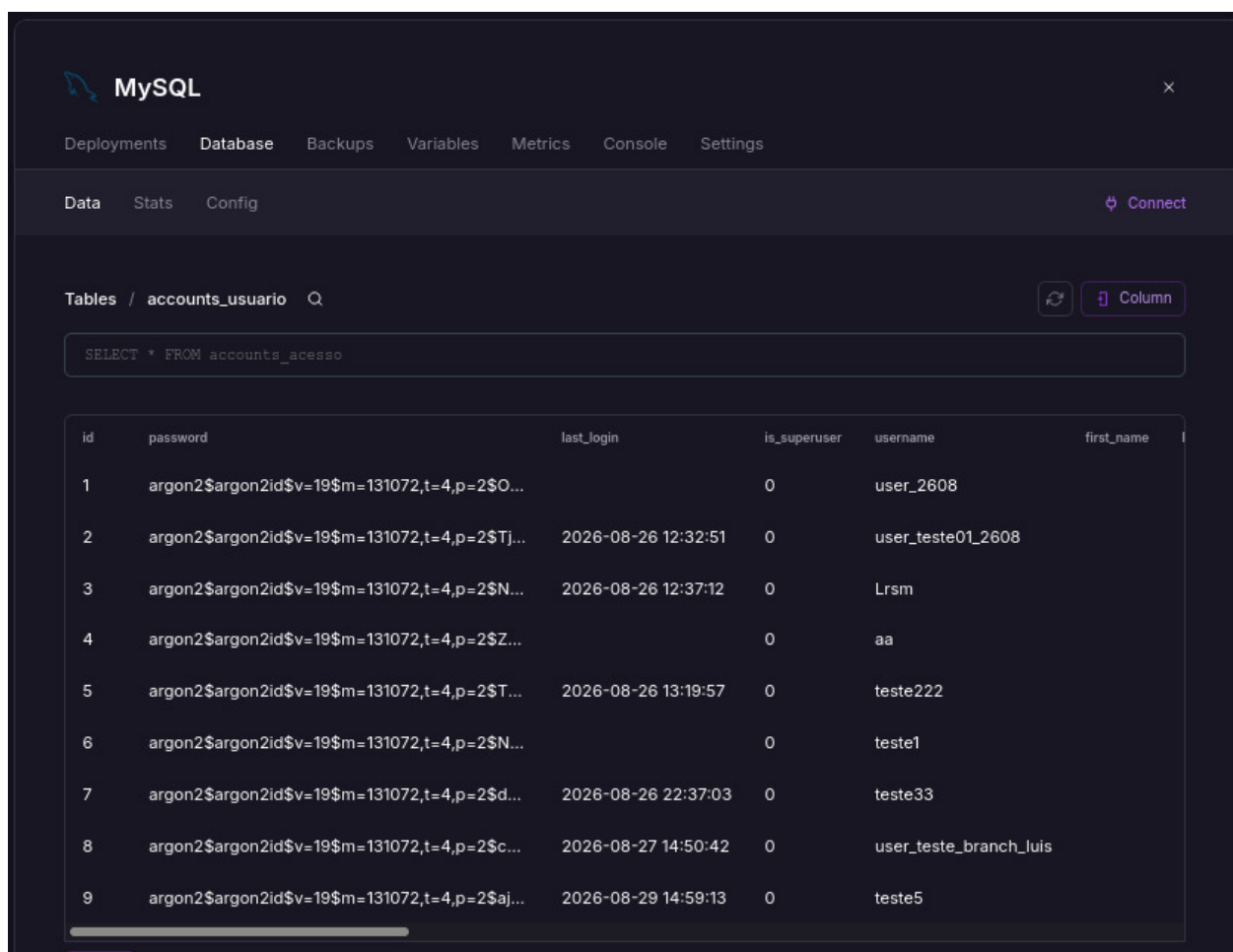
Implementação: O sistema utiliza o algoritmo Argon2 para o hash das senhas. A dependência `argon2-cffi==25.1.0` foi adicionada e a classe `Argon2PasswordHasher` foi herdada e customizada no `hashers.py`

Justificativa técnica: O Argon2 é o algoritmo recomendado por ser resistente a ataques de força bruta utilizando GPUs.

Evidência:

Trecho de configuração em `hashers.py`

Representação no banco de dados da coluna password



id	password	last_login	is_superuser	username	first_name
1	argon2\$argon2id\$v=19\$m=131072,t=4,p=2\$O...		0	user_2608	
2	argon2\$argon2id\$v=19\$m=131072,t=4,p=2\$Tj...	2026-08-26 12:32:51	0	user_teste01_2608	
3	argon2\$argon2id\$v=19\$m=131072,t=4,p=2\$N...	2026-08-26 12:37:12	0	Lrsm	
4	argon2\$argon2id\$v=19\$m=131072,t=4,p=2\$Z...		0	aa	
5	argon2\$argon2id\$v=19\$m=131072,t=4,p=2\$T...	2026-08-26 13:19:57	0	teste222	
6	argon2\$argon2id\$v=19\$m=131072,t=4,p=2\$N...		0	teste1	
7	argon2\$argon2id\$v=19\$m=131072,t=4,p=2\$d...	2026-08-26 22:37:03	0	teste33	
8	argon2\$argon2id\$v=19\$m=131072,t=4,p=2\$c...	2026-08-27 14:50:42	0	user_teste_branch_luis	
9	argon2\$argon2id\$v=19\$m=131072,t=4,p=2\$aj...	2026-08-29 14:59:13	0	teste5	

Figura 1 Hash da senha armazenado utilizando o algoritmo Argon2 no banco de dados.

Resultado: ATENDIDO

1.2 Parâmetros de custo do hash configurados e justificados

Implementação: Foi criada a classe CustoHash que especifica `time_cost = 4`, `memory_cost = 128 1024` e `parallelism = 2`

Justificativa técnica: Estes parâmetros definem a complexidade (4 iterações), a exigência de memória (128 MB RAM) e o nível de paralelismo (2 threads), garantindo resistência adequada contra cálculos acelerados sem sobrecarregar o servidor.

Evidência:

MySQL

Deployments Database Backups Variables Metrics Console Settings

Data Stats Config [Connect](#)

Tables / accounts_usuario

SELECT * FROM accounts_usuario

id	password	last_login	is_superuser	username	first_name
1	argon2\$sargon2id\$v=19\$m=131072,t=4,p=2\$O...		0	user_2608	
2	argon2\$sargon2id\$v=19\$m=131072,t=4,p=2\$Tj...	2026-08-26 12:32:51	0	user_teste01_2608	
3	argon2\$sargon2id\$v=19\$m=131072,t=4,p=2\$N...	2026-08-26 12:37:12	0	Lrsm	
4	argon2\$sargon2id\$v=19\$m=131072,t=4,p=2\$Z...		0	aa	
5	argon2\$sargon2id\$v=19\$m=131072,t=4,p=2\$T...	2026-08-26 13:19:57	0	teste222	
6	argon2\$sargon2id\$v=19\$m=131072,t=4,p=2\$N...		0	teste1	
7	argon2\$sargon2id\$v=19\$m=131072,t=4,p=2\$d...	2026-08-26 22:37:03	0	teste33	
8	argon2\$sargon2id\$v=19\$m=131072,t=4,p=2\$c...	2026-08-27 14:50:42	0	user_teste_branch_luis	
9	argon2\$sargon2id\$v=19\$m=131072,t=4,p=2\$aj...	2026-08-29 14:59:13	0	teste5	

Hash gerado mostrando m=131072,t=4,p=2 no banco de dados (Figura 1).

Resultado: ATENDIDO

1.3 Uso de salt criptográfico único por usuário

Implementação: O Argon2 utilizado no Django (e instanciado como CustoHash) gera nativa e automaticamente um salt criptográfico distinto e aleatório para cada cadastro.

Justificativa técnica: O salt único para cada senha impede ataques que utilizam tabelas pré-computadas (Rainbow Tables).

id	password	last_login	is_superuser	username	first_name
1	argon2\$argon2id\$v=19\$m=131072,t=4,p=2\$O...		0	user_2608	
2	argon2\$argon2id\$v=19\$m=131072,t=4,p=2\$Tj...	2026-08-26 12:32:51	0	user_teste01_2608	
3	argon2\$argon2id\$v=19\$m=131072,t=4,p=2\$N...	2026-08-26 12:37:12	0	Lrsm	
4	argon2\$argon2id\$v=19\$m=131072,t=4,p=2\$Z...		0	aa	
5	argon2\$argon2id\$v=19\$m=131072,t=4,p=2\$T...	2026-08-26 13:19:57	0	teste222	
6	argon2\$argon2id\$v=19\$m=131072,t=4,p=2\$N...		0	teste1	
7	argon2\$argon2id\$v=19\$m=131072,t=4,p=2\$d...	2026-08-26 22:37:03	0	teste33	
8	argon2\$argon2id\$v=19\$m=131072,t=4,p=2\$c...	2026-08-27 14:50:42	0	user_teste_branch_luis	
9	argon2\$argon2id\$v=19\$m=131072,t=4,p=2\$aj...	2026-08-29 14:59:13	0	teste5	

Evidência: Verificando o print da tabela `accounts_usuario` (Figura 1), cada registro da string `argon2$argon2id$...` apresenta conjuntos aleatórios após a definição dos parâmetros, os quais representam o salt exclusivo.

Resultado: ATENDIDO

1.4 Armazenamento correto do hash + salt

Implementação: O framework Django gerencia a string resultante do Argon2 gravando nativamente o identificador do algoritmo, parâmetros, salt e o hash concatenados em um formato padronizado seguro e salvando-o no campo `password`

Justificativa técnica: Manter este formato garante que o sistema consiga extrair o salt e os parâmetros exatos necessários na etapa de autenticação.

Evidência: O armazenamento na tabela `accounts_usuario` (Figura 1) demonstra o formato correto `algoritmo$versão$parâmetros$salt$hash`

MySQL

Deployments Database Backups Variables Metrics Console Settings

Data Stats Config [Connect](#)

Tables / accounts_usuario [Column](#)

SELECT * FROM accounts_usuario

id	password	last_login	is_superuser	username	first_name
1	argon2\$argon2id\$v=19\$m=131072,t=4,p=2\$O...		0	user_2608	
2	argon2\$argon2id\$v=19\$m=131072,t=4,p=2\$Tj...	2026-08-26 12:32:51	0	user_teste01_2608	
3	argon2\$argon2id\$v=19\$m=131072,t=4,p=2\$N...	2026-08-26 12:37:12	0	Lrsm	
4	argon2\$argon2id\$v=19\$m=131072,t=4,p=2\$Z...		0	aa	
5	argon2\$argon2id\$v=19\$m=131072,t=4,p=2\$T...	2026-08-26 13:19:57	0	teste222	
6	argon2\$argon2id\$v=19\$m=131072,t=4,p=2\$N...		0	teste1	
7	argon2\$argon2id\$v=19\$m=131072,t=4,p=2\$d...	2026-08-26 22:37:03	0	teste33	
8	argon2\$argon2id\$v=19\$m=131072,t=4,p=2\$c...	2026-08-27 14:50:42	0	user_teste_branch_luis	
9	argon2\$argon2id\$v=19\$m=131072,t=4,p=2\$aj...	2026-08-29 14:59:13	0	teste5	

Resultado: ATENDIDO

1.5 Autenticação de dois fatores (2FA) implementada

Implementação: A autenticação em duas etapas baseada em TOTP foi integrada utilizando a biblioteca django-otp. O usuário acessa a página `setup_2fa` para configurar o aplicativo de autenticação (ex.: Google Authenticator) lendo o QR Code disponibilizado e inserindo o código gerado.

Justificativa técnica: A autenticação 2FA baseada em TOTP acrescenta a necessidade do usuário "ter algo" (o dispositivo físico gerador do código), reduzindo a ameaça caso a senha seja comprometida.

Evidência:

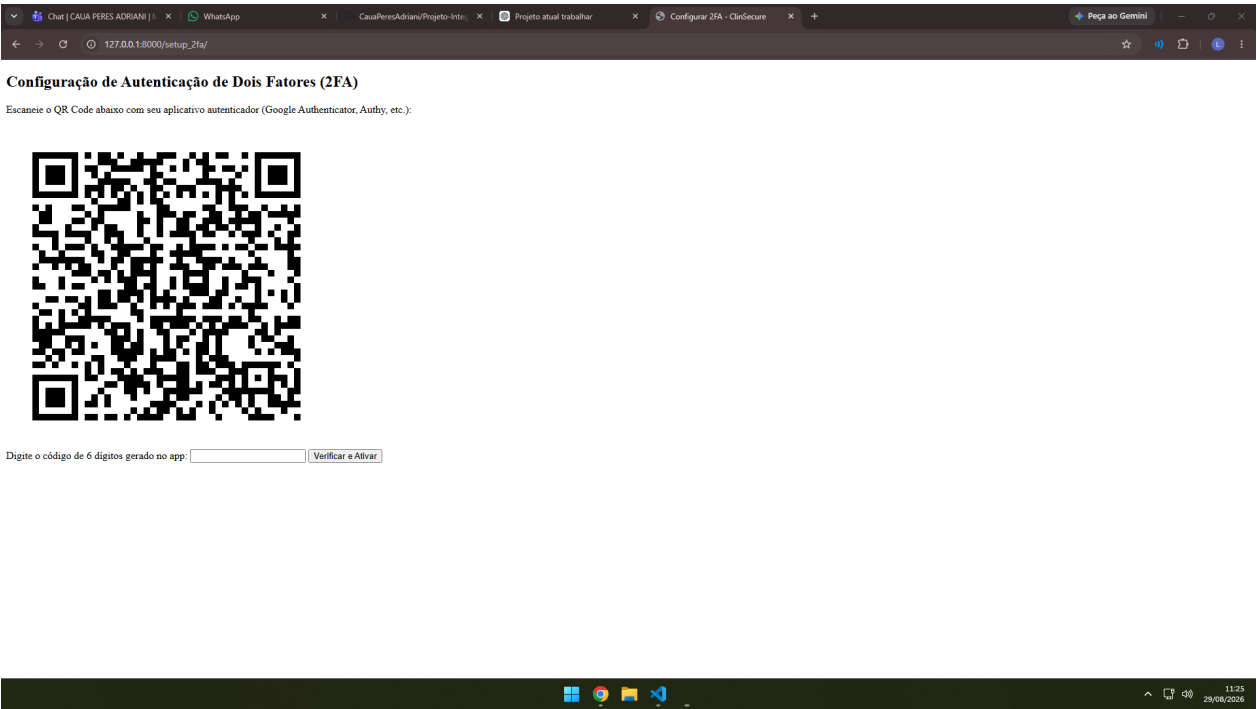


Figura 2 QR Code gerado para configuração do 2FA pelo usuário.

MySQL

Deployments Database Backups Variables Metrics Console Settings

Data Stats Config

Tables / otp_totp_totpdevice

SELECT * FROM accounts_acesso

id	name	confirmed	key	step	t0	digits	tolerance	drift	last
1	Celular Principal	0	dbb8c99c2ebbc079fffd295b40afabf8e0739f56	30	0	6	1	0	-1
2	Celular Principal	1	02a0cb28d0211ce74a3703fec85fd25756f72929	30	0	6	1	0	55
3	Celular Principal	1	0d18b9d87e19acd498bef4d5a634a5fa4367810c	30	0	6	1	0	55
4	Celular Principal	0	59633e02b4de7b8d7902faab5a598fdbb1cef5...	30	0	6	1	0	-1
5	Celular Principal	1	62d76de1d814c1e04d62d6b90f2e65ff816f8348	30	0	6	1	0	55
6	Celular Principal	0	b9826af62d30bcffff52ffe54d6f5997da9ee6da	30	0	6	1	0	-1
7	Celular Principal	1	459ffecfb90142c68a4e667db7235372b017f368	30	0	6	1	0	55
8	Celular Principal	1	5c6b8d0d68a0076d4e8aa2036d709cc2811de...	30	0	6	1	0	55
10	Celular Principal	1	0ab7d66b9be299f2c0794077286c416ee4875...	30	0	6	1	2	55

+ Row

Figura 3 Dispositivos TOTP cadastrados na tabela otp_totp_totpdevice.

Resultado: ATENDIDO

1.6 Validação do 2FA após autenticação primária

Implementação: Durante o login, após validar com sucesso usuário e senha, a sessão não é concluída. Em vez disso, é registrada a validação provisória no banco (auditlog), e ocorre redirecionamento para `verificar_2fa_view`, onde o usuário é forçado a preencher o token de 6 dígitos.

Justificativa técnica: Interromper a etapa da sessão assegura que a área restrita do sistema permaneça inacessível até a conclusão da dupla verificação.

Evidência:

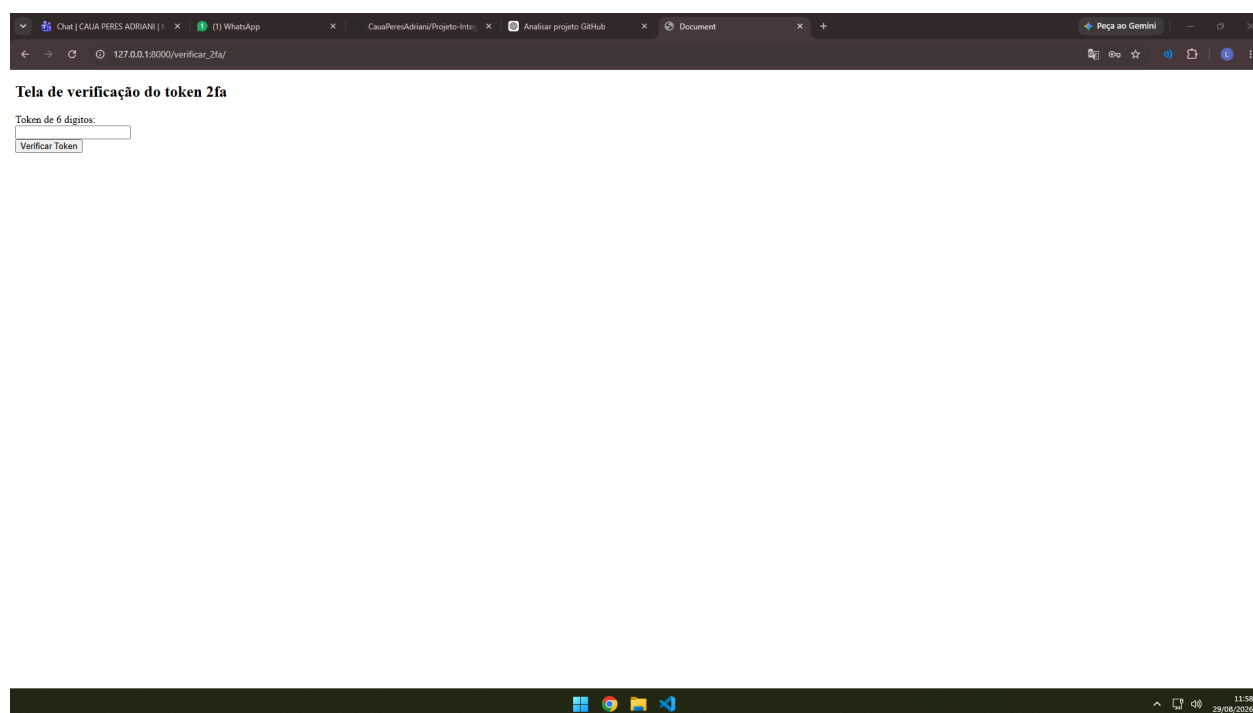


Figura 4 Tela de verificação exigindo o código de 6 dígitos.

Resultado: ATENDIDO

1.7 Fluxo de autenticação documentado

Implementação: O fluxo está documentado detalhadamente neste relatório técnico (vide Seção 3 e seções individuais).

Justificativa técnica: Garantir manutenção futura do projeto.

Evidência: A presente documentação estruturada com lógicas comentadas no código-fonte em arquivos como `views.py` e também na documentação do fluxo de autenticação

Resultado: ATENDIDO

1.8 Evidências funcionais (prints, logs ou testes)

Implementação: Realização e captura de todos os testes pelo front-end exigidos pelas instruções do professor.

Justificativa técnica: Prova documental que atende diretamente à Diretriz "Testes via Front-end".

Evidência: As imagens disponibilizadas neste documento demonstram sucesso nos fluxos (Seção 5 e anexos).

Resultado: ATENDIDO

1.9 Sessões com tempo de expiração

Implementação: As diretivas `SESSION_COOKIE_AGE = 900` e `SESSION_EXPIRE_AT_BROWSER_CLOSE = True` foram inseridas no `settings.py` para aplicar 15 minutos (900 segundos) de tempo máximo.

Justificativa técnica: Expirar sessões mitiga o risco de ataques baseados em computadores abandonados com usuários logados.

Evidência: video de demonstração do fluxo de autenticação

Resultado:

1.10 Invalidação de sessão no logout

Implementação: A funcionalidade é tratada em `meu_logout_view(request)` e pela inserção de `SESSION_SAVE_EVERY_REQUEST = True`

Justificativa técnica: Invoca a destruição do cookie e limpa variáveis locais no servidor, impedindo reuso.

Evidência: Home page contendo "Sair do Sistema" e código `logout_view` Implementado no código, porém sem evidência funcional anexada.

1.11 Proteção contra força bruta (rate limit, bloqueio, atraso)

Implementação: A função `meu_login_view` checa a variável `bloqueado_ate` contra `timezone.now()` Foram registrados eventos de log sinalizando falhas contínuas que resultam em bloqueio do 2FA. As telas de front-end exibem mensagens do tipo "Restam X tentativas(s)".

Justificativa técnica: Responde a ataques automatizados diminuindo a velocidade de processamento ou interrompendo temporariamente o acesso do ofensor (Rate Limit / Bloqueio).

Evidência:

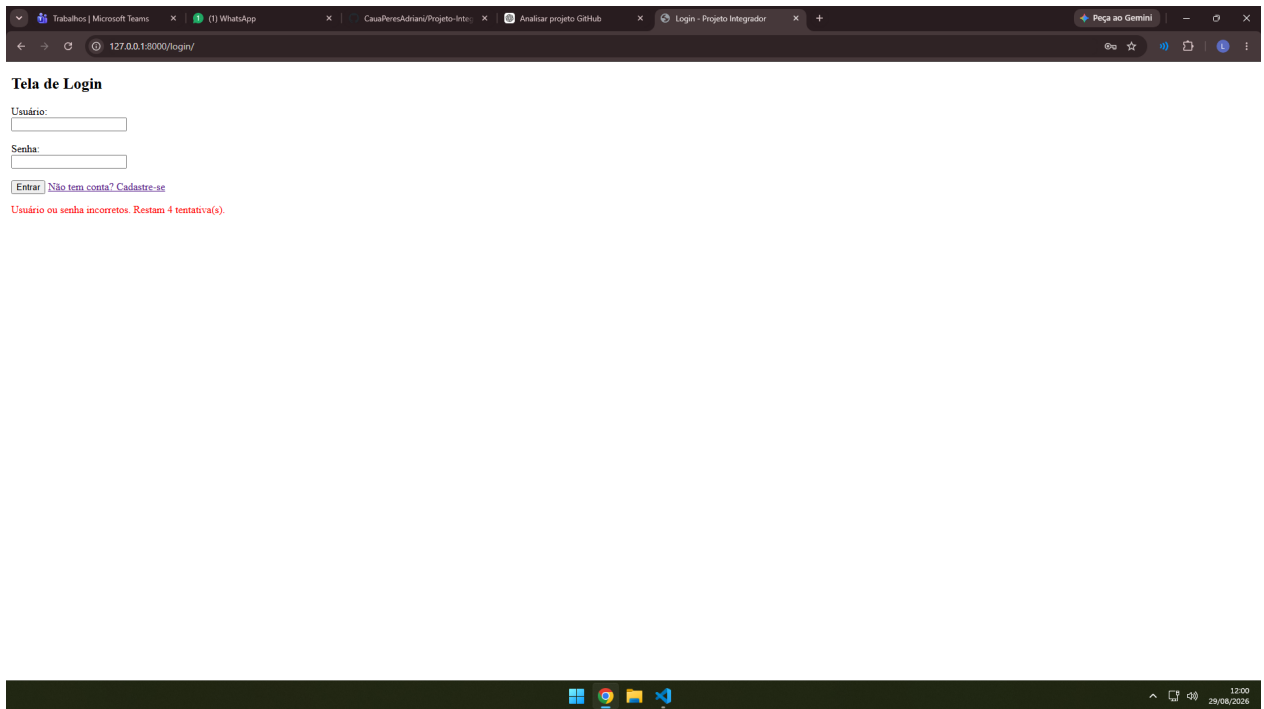


Figura 5 Erro de senha no login: "Usuário ou senha incorretos. Restam 4 tentativa(s)."

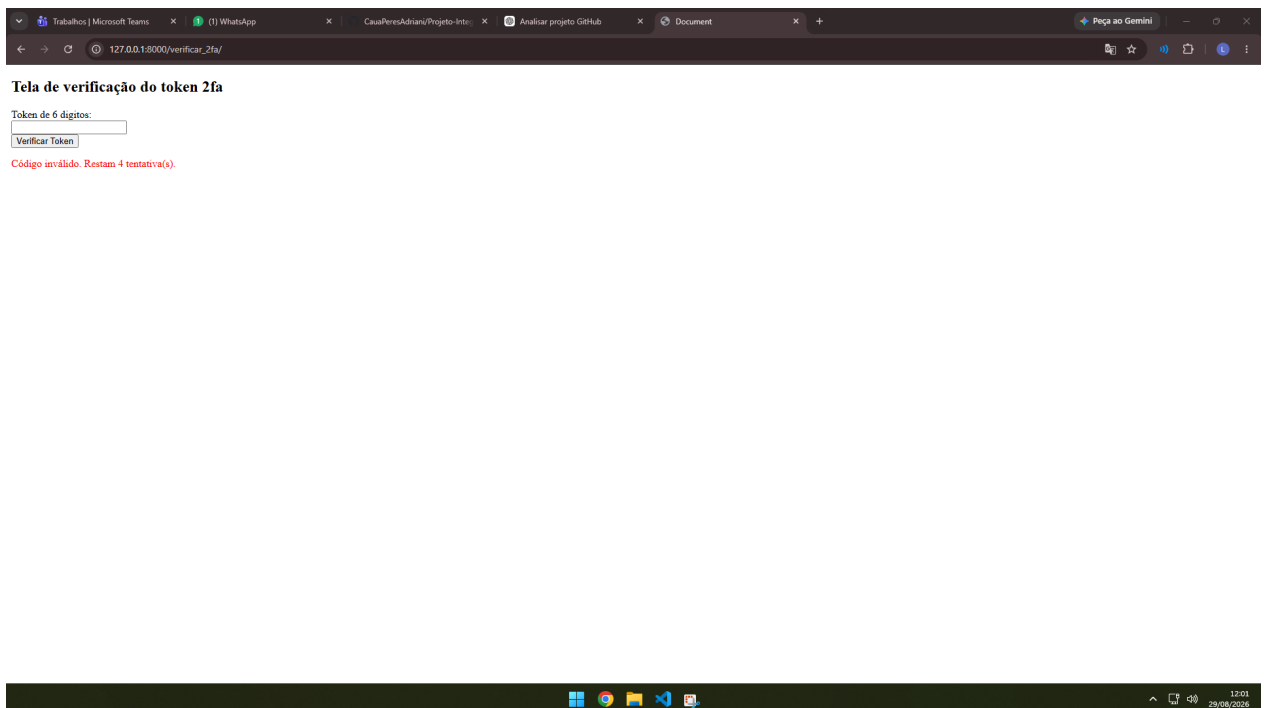


Figura 6 Erro de 2FA: "Código inválido. Restam 4 tentativa(s)."

id	evento	data_hora	ip	resultado	detalhes	us
1	Login bem-sucedido	2026-08-27 14:49:36	127.0.0.1	Sucesso	Usuário logado com sucesso.	8
2	2FA	2026-08-27 14:50:01	127.0.0.1	Falha	Código 2FA incorreto. Tentativa 1.	8
3	2FA	2026-08-27 14:50:20	127.0.0.1	Falha	Código 2FA incorreto. Tentativa 2.	8
4	Senha validada	2026-08-29 14:20:29	127.0.0.1	Sucesso	Usuário e senha validados. Aguardando valida...	7
5	2FA	2026-08-29 14:21:06	127.0.0.1	Falha	Código 2FA incorreto. Tentativa 5.	7
6	Bloqueio 2FA	2026-08-29 14:21:11	127.0.0.1	Bloqueado	2FA bloqueado após 5 tentativas incorretas.	7
7	Senha validada	2026-08-29 14:25:02	127.0.0.1	Sucesso	Usuário e senha validados. Aguardando valida...	9
8	Senha validada	2026-08-29 14:53:55	127.0.0.1	Sucesso	Usuário e senha validados. Aguardando valida...	9
9	Senha validada	2026-08-29 14:58:17	127.0.0.1	Sucesso	Usuário e senha validados. Aguardando valida...	9
10	Login bem-sucedido	2026-08-29 14:59:13	127.0.0.1	Sucesso	Login concluído após validação do usuário, se...	9

Print dos logs do banco provando a aplicação do bloqueio consta na Seção 8.

Resultado: ATENDIDO

1.12 Justificativas técnicas documentadas

Implementação: Incluídas de forma unificada neste documento.

Resultado: ATENDIDO

5. Evidências Funcionais

Esta seção destina-se a consolidar os testes práticos requisitados pelo professor, executados via interface do usuário.

Cadastro de usuário

Objetivo/Procedimento: Criar usuário utilizando interface.

Resultado Obtido: Tela carregada corretamente.

Evidência:

Tela de Cadastro ClinSecure

Usuário:

Senha:

Email:

CPF:

Perfil:

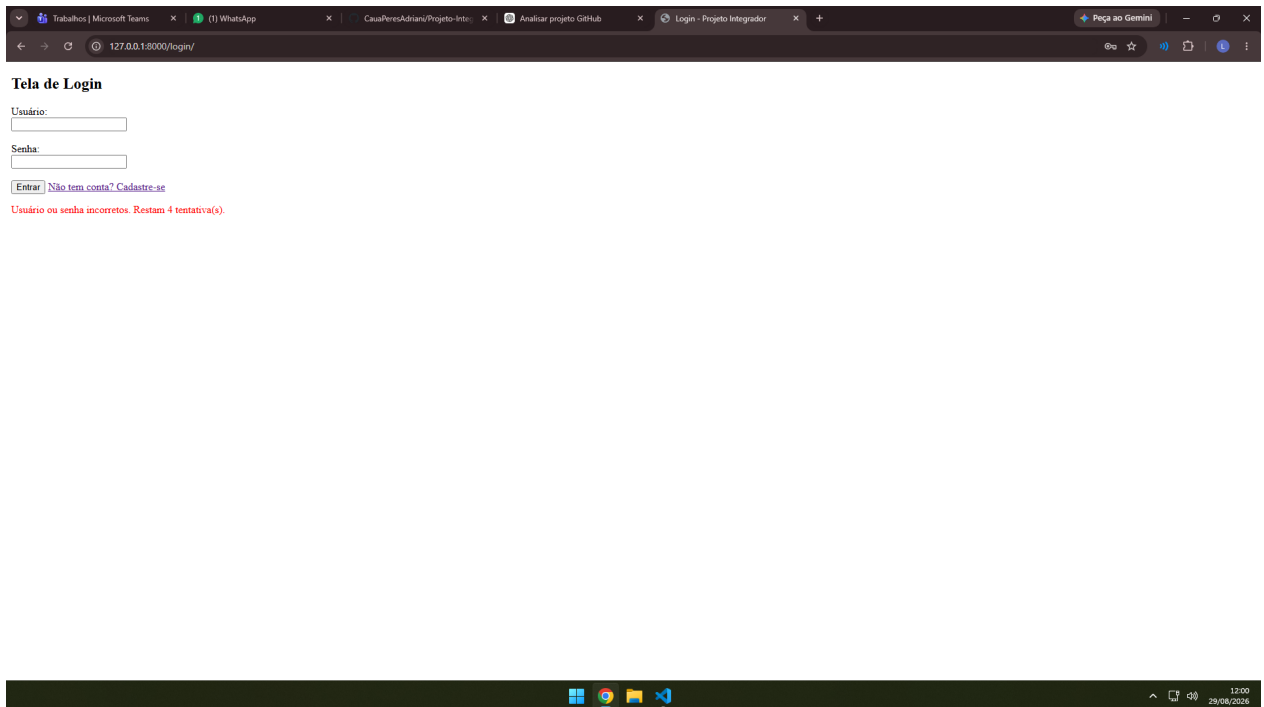
[Cadastrar](#) [Já tem conta? Faça login](#)

Login com credenciais inválidas e Proteção contra Força Bruta

Objetivo/Procedimento: Inserir falhas no usuário e monitorar contador.

Resultado Obtido: O sistema relata falha e decrementa tentativas (Restam 4).

Evidência:

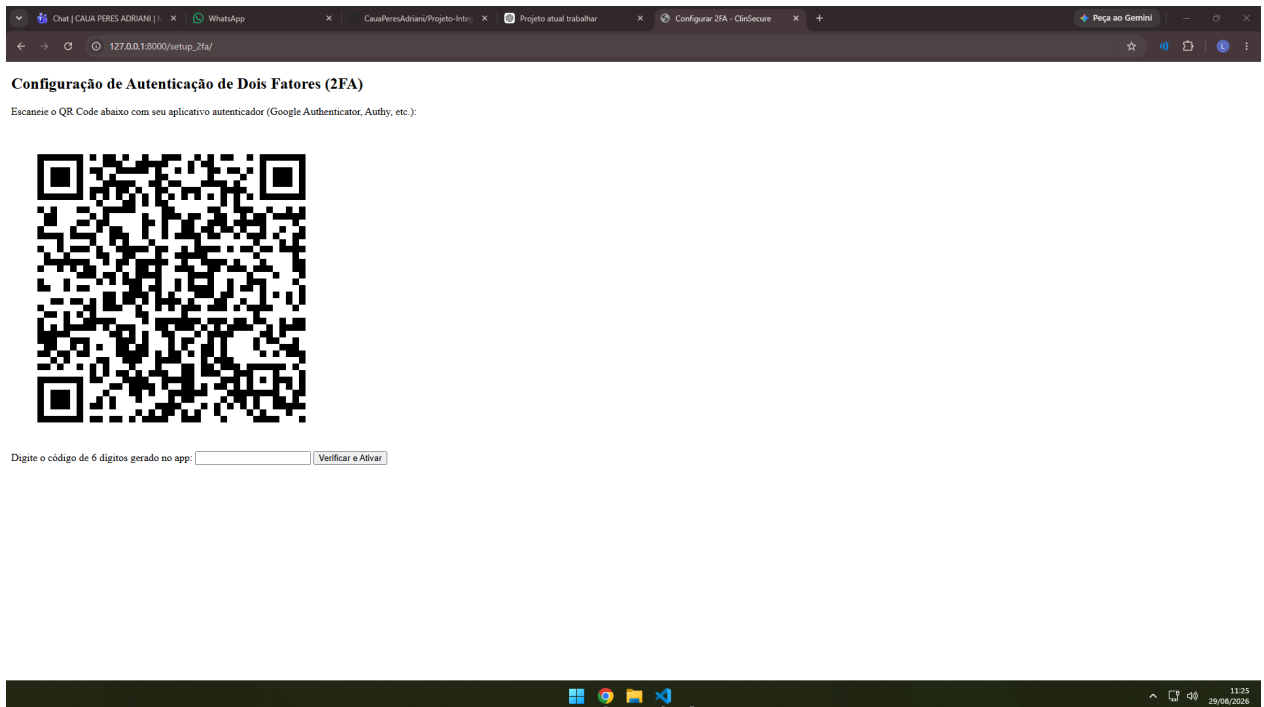


Configuração do 2FA

Objetivo/Procedimento: Associar código via aplicativo.

Resultado Obtido: QR code gerado para leitura.

Evidência:

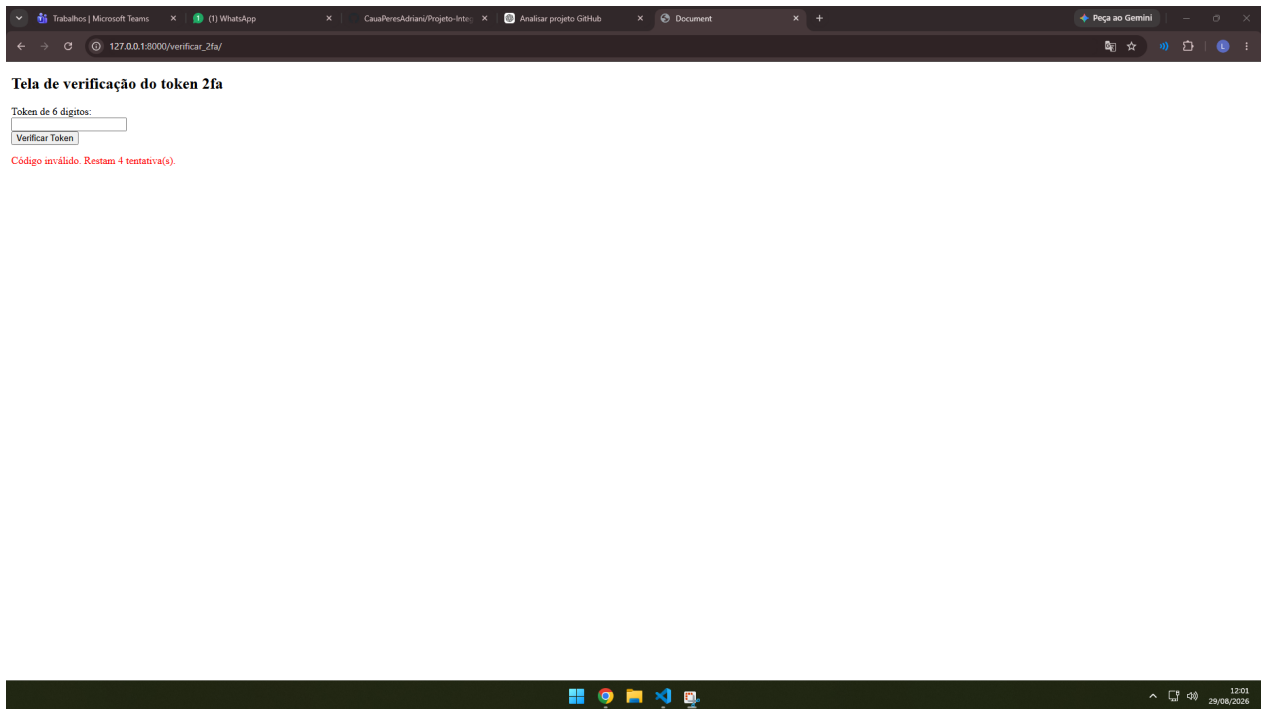


Validação de 2FA Inválido

Objetivo/Procedimento: Errar intencionalmente token após logar.

Resultado Obtido: Avisos decrementais: "Código inválido. Restam 4 tentativa(s)".

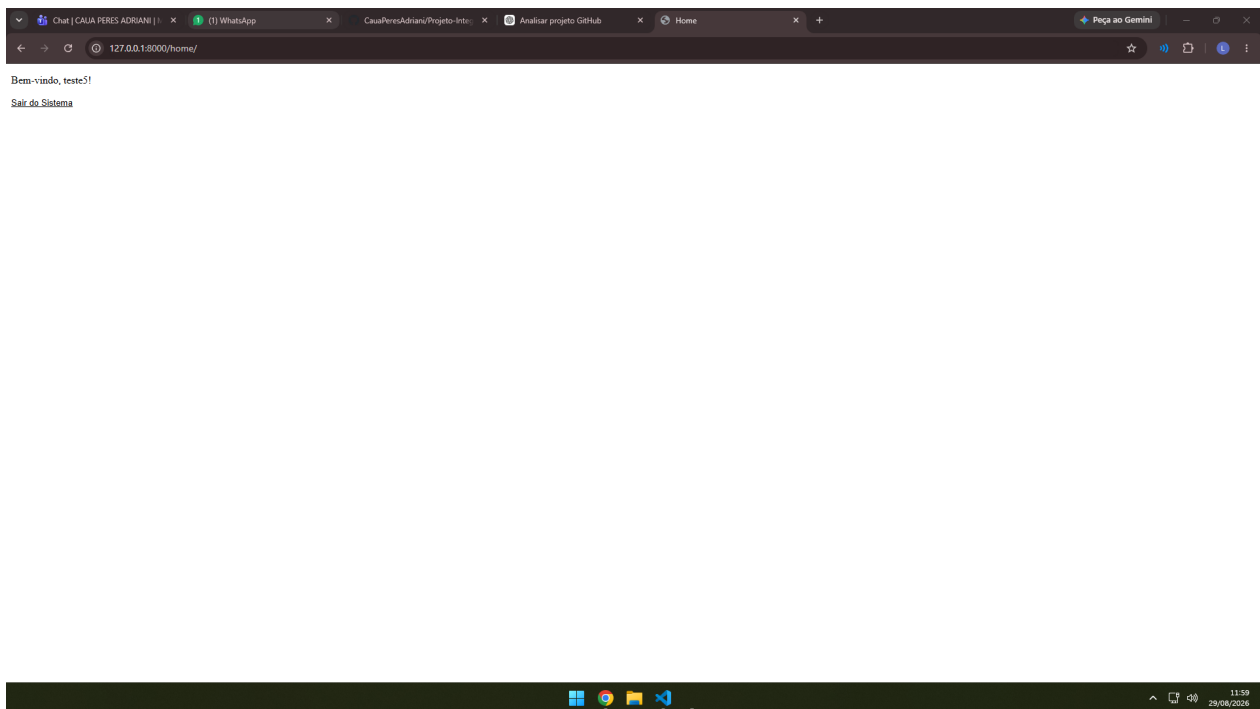
Evidência:



Acesso à página protegida

Objetivo/Procedimento: Teste de sucesso completo no acesso.

Resultado Obtido: Visualização da Home "Bem-vindo, teste5!".



Evidência:

Bloqueio após cinco tentativas / Tentativa de acesso após logout / Atraso Progressivo:

6. Evidências do Banco de Dados

Representação do Argon2 e Usuários: Os hashes contendo algoritmos, parâmetros, salt e criptografia estão na tabela `accounts_usuario` na coluna `password` (Vide Figura 1). As tabelas jamais expõem tokens puros (os logs tratam apenas como sucessos ou falhas).

Controle TOTP: Tokens provisionados gravados em `otp_totp_totpdevice` demonstrando devices vinculados ativamente ou com o flag `confirmed`.

7. Evidências de Código

Os arquivos apresentados demonstram o preenchimento de todos os requisitos:

`hashers.py`: Estabelece as regras fortes de `Argon2PasswordHasher` com custo de tempo (4) e memória (128 KB).

`settings.py`: Parametriza limites das sessões (`SESSION_COOKIE_AGE`), segurança baseada em Cookies (`HTTPONLY`), configuração de Apps de segurança OTP (`django_otp`) e chamadas de rotas de login.

models.py: Abstração do modelo customizado em Usuario(AbstractUser), criação e estruturação de tabelas para a criação de AuditLog com armazenamento de rastros completos por IP.

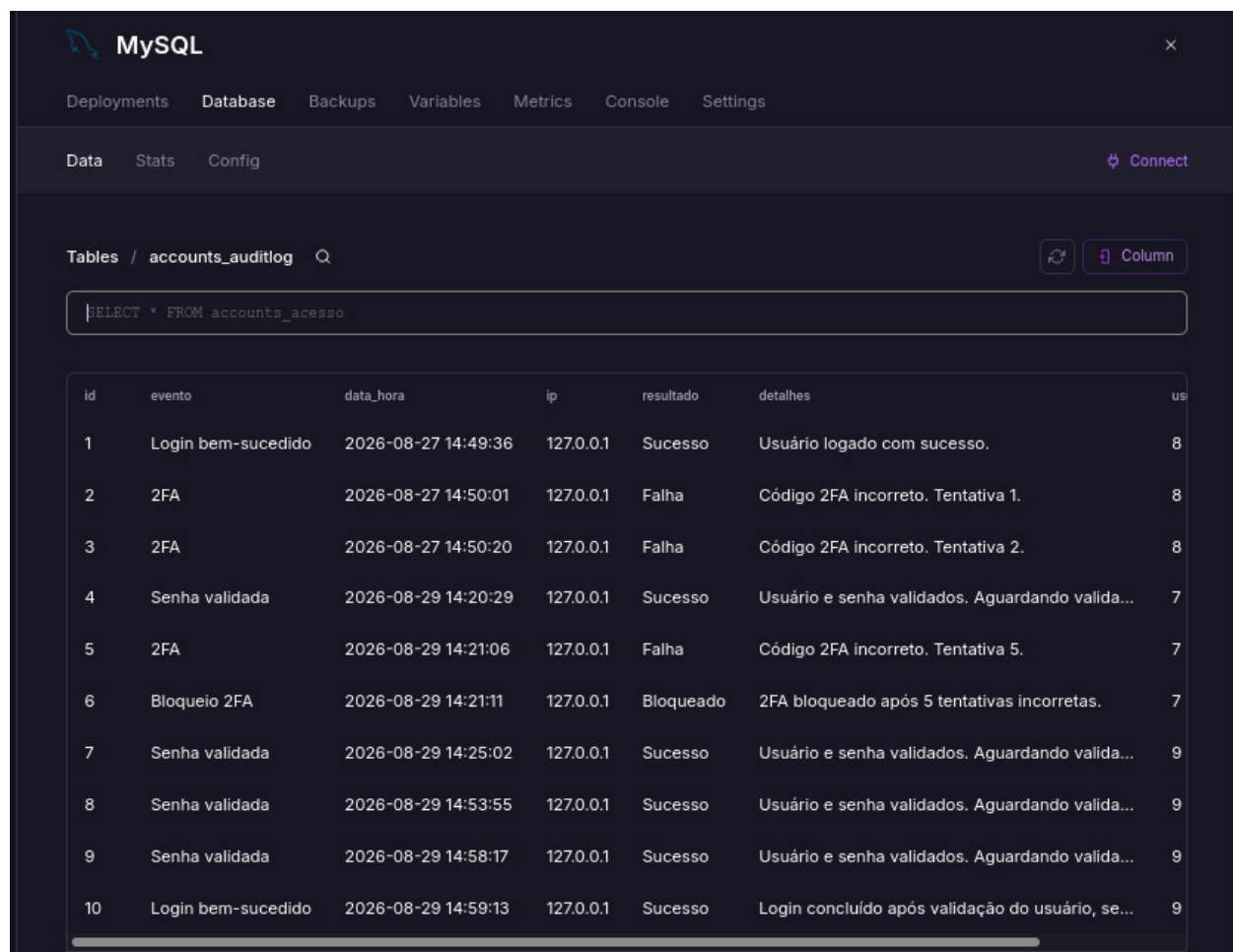
views.py: Onde as regras de negócio de bloqueado_ate > agora, e transições meu_login_view - > setup_2fa -> verificar_2fa são validadas em produção.

8. Auditoria e Logs

As ações do usuário possuem rastreabilidade na tabela dedicada AuditLog. A evidência revela:

Eventos registrados: Login bem-sucedido (ID 1), Falhas de TOTP (ID 2, 3), Senha primária verificada e aguardando token (ID 4, 7, 8, 9).

Evidência de Bloqueio (Força bruta 2FA): O ID 6 no banco demonstra categoricamente um registro classificado como "Bloqueado" nos detalhes: "2FA bloqueado após 5 tentativas incorretas."



MySQL

Deployments Database Backups Variables Metrics Console Settings

Data Stats Config

Tables / accounts_auditlog

SELECT * FROM accounts_acesso

id	evento	data_hora	ip	resultado	detalhes	us
1	Login bem-sucedido	2026-08-27 14:49:36	127.0.0.1	Sucesso	Usuário logado com sucesso.	8
2	2FA	2026-08-27 14:50:01	127.0.0.1	Falha	Código 2FA incorreto. Tentativa 1.	8
3	2FA	2026-08-27 14:50:20	127.0.0.1	Falha	Código 2FA incorreto. Tentativa 2.	8
4	Senha validada	2026-08-29 14:20:29	127.0.0.1	Sucesso	Usuário e senha validados. Aguardando valida...	7
5	2FA	2026-08-29 14:21:06	127.0.0.1	Falha	Código 2FA incorreto. Tentativa 5.	7
6	Bloqueio 2FA	2026-08-29 14:21:11	127.0.0.1	Bloqueado	2FA bloqueado após 5 tentativas incorretas.	7
7	Senha validada	2026-08-29 14:25:02	127.0.0.1	Sucesso	Usuário e senha validados. Aguardando valida...	9
8	Senha validada	2026-08-29 14:53:55	127.0.0.1	Sucesso	Usuário e senha validados. Aguardando valida...	9
9	Senha validada	2026-08-29 14:58:17	127.0.0.1	Sucesso	Usuário e senha validados. Aguardando valida...	9
10	Login bem-sucedido	2026-08-29 14:59:13	127.0.0.1	Sucesso	Login concluído após validação do usuário, se...	9

Figura 7 Registros de auditoria exibindo o rastreamento exato do bloqueio após falhas contínuas.

Importância: Sem estes registros precisos que envolvem IP e timestamps precisos, não haveria responsabilidade transparente em falhas de acesso na gerência de pesquisa clínica.

9. Sessões e Logout

Configurações de Segurança do Cookie: Encontradas em settings.py O cookie tem proteção ativa com `SESSION_COOKIE_HTTPONLY = True`, impedindo acesso indevido via scripts, além de `SESSION_COOKIE_SAMESITE = 'Lax'`

Tempo de sessão: Configurada para 900 segundos (15 minutos) e `SESSION_EXPIRE_AT_BROWSER_CLOSE = True`

Autenticação e Logout: A classe `@login_required` protege a view da home para deslogados, enquanto a função `meu_logout_view(request)` gerencia o encerramento da conexão.

Nota de Ausência: Falta documentar com capturas o teste de tentar acesso direto ao sistema através do navegador, pós-logout.

10. Histórico de Desenvolvimento

A gestão e desenvolvimento através de versionamento encontram-se no repositório com contribuições dos responsáveis: Cauã Peres Adriani e Luis Poli. O fluxo ocorreu organizadamente por meio de diferentes Pull Requests temáticos de segurança (exemplos em logs: fix/login-2fa, feat/ratelimit, feat(auth): implementar fluxo completo...).

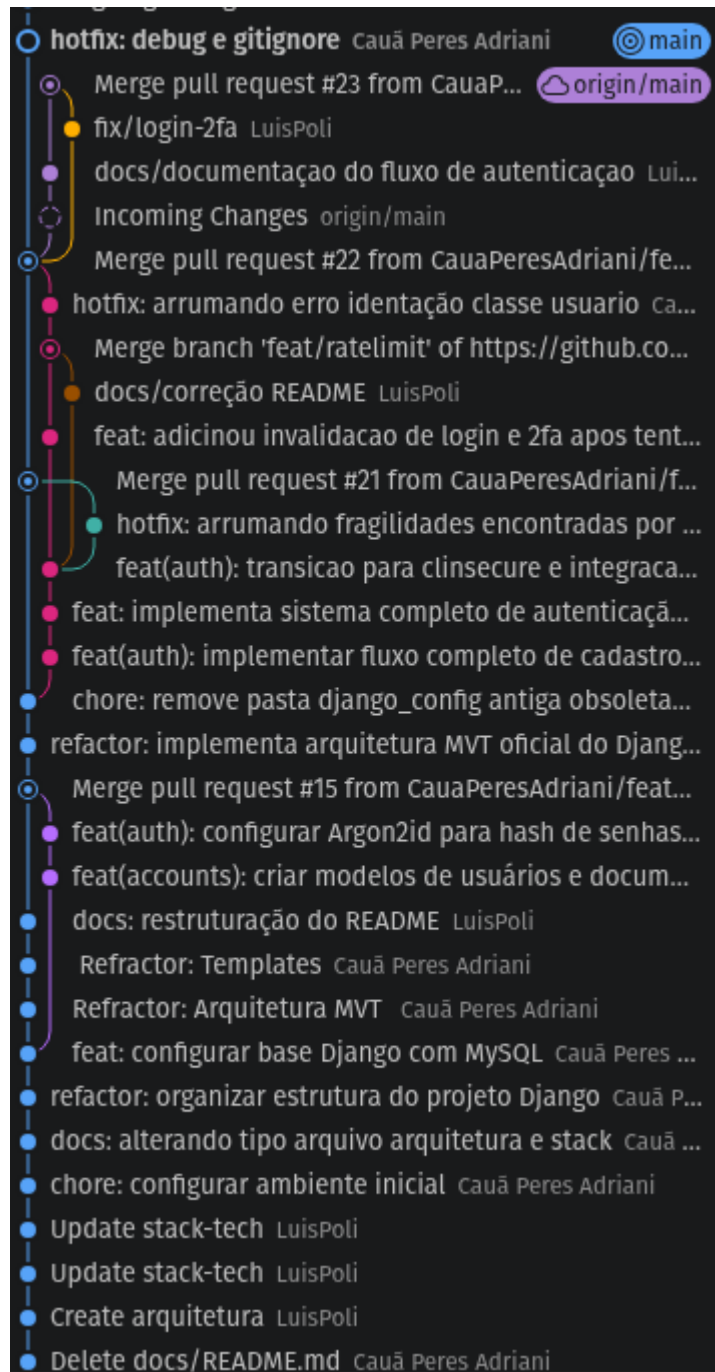


Figura 8 Histórico e ramificação (branches) das entregas.

11. Kanban

O trabalho da equipe foi fracionado no painel Kanban alocado dentro do GitHub (Projetos).

Divisão: É nítido o avanço nas guias Backlog, In review e Done.

Tarefas (Exemplos): "Armazenamento correto do hash + salt" (Done), "Autenticação de dois fatores" (Done), "Sessões com tempo de expiração" (In review) e tarefas futuras "Funcionalidade de recuperação de senha" (Backlog).

Responsáveis: A equipe utilizou designações com fotos/ícones para alocação mútua no andamento (Cauã e Luis).

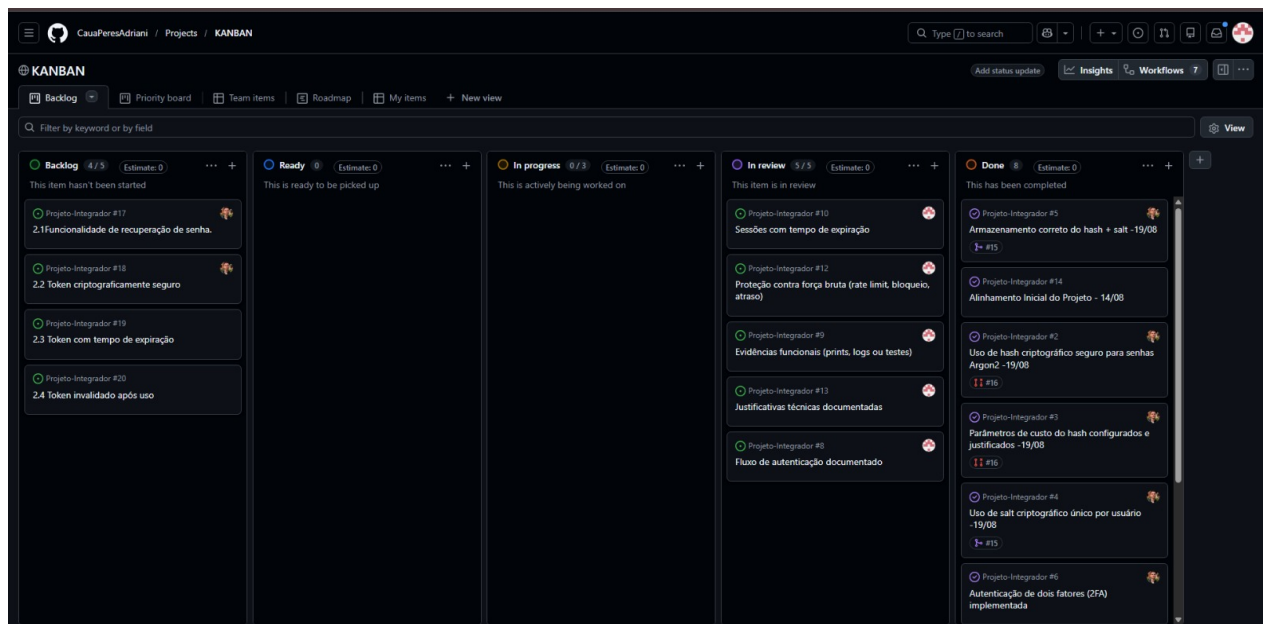


Figura 9 Quadro Kanban detalhado, comprovando planejamento e execução das tarefas.

12. Repositório GitHub

O repositório principal do projeto (Visão na "main" branch) armazena código, arquivos .gitignore, documento LICENSE e manage.py. Todo sistema de dependências se encontra organizado em um requirements.txt estruturado (onde a dependência argon2-cffi é chamada), além da existência de documentação no README.md e da pasta docs/ mantendo as anotações pertinentes à arquitetura.

14. Checklist Final

Nº	Requisito	X
1. Autenticação e Gestão de Credenciais		
1.1	Uso de hash criptográfico seguro para senhas (Argon2, bcrypt ou PBKDF2)	☒
1.2	Parâmetros de custo do hash configurados e justificados	☒
1.3	Uso de <i>salt</i> criptográfico único por usuário	☒
1.4	Armazenamento correto do hash + salt	☒
1.5	Autenticação de dois fatores (2FA) implementada	☒
1.6	Validação do 2FA após autenticação primária	☒
1.7	Fluxo de autenticação documentado	☒
1.8	Evidências funcionais (prints, logs ou testes)	☒
1.9	Sessões com tempo de expiração	☒
1.10	Invalidação de sessão no logout	☒
1.11	Proteção contra força bruta (rate limit, bloqueio, atraso)	☒
1.12	Justificativas técnicas documentadas	☒
2. Recuperação de Senha		
2.1	Funcionalidade de recuperação de senha implementada	☐
2.2	Token criptograficamente seguro	☐
2.3	Token com tempo de expiração	☐
2.4	Token invalidado após uso	☐
2.5	Falha correta para token expirado	☐

